# SOCIAL NETWORKS

NPTEL Online Course • Prof. Sudarshan Iyengar, IIT Ropar

**Week 9 — Question Paper (Comprehension-Based MCQ / MSQ)**

## Comprehension 1 — Why a Few Channels Dominate: Power Law

### Case Study

An analyst, Anjali, studies a video-creator platform. She builds a directed graph in which each channel is a node and a recommendation from one channel to another is an edge, then plots the degree distribution along the x-axis the in-degree (how many channels recommend a given channel) and along the y-axis the fraction of channels having that in-degree. She wants to know what shape this plot takes.

As a baseline she first imagines channels recommending each other purely at random. That model gives a familiar hump: most channels sit near an average in-degree, with few channels far below or far above. The real platform, however, looks nothing like that hump. Its plot starts high on the left and falls away steeply, then trails off to the right in a long, thin tail meaning a great many channels with very small in-degree and a small handful of channels with enormous in-degree. These few enormous-degree channels are the platform's hubs.

To explain the shape Anjali turns to a growth story: the platform adds channels over time, and each newcomer is more likely to be recommended by, and to follow, channels that are already widely recommended a self-reinforcing advantage. She also recalls a simple graphical test for confirming the pattern, and notes a practical consequence: a network built this way leans heavily on its hubs, which matters when one asks how the network holds up if nodes are removed.

### Questions 1–10

**Q1.** [MCQ] How does the real platform's degree distribution differ from the random-recommendation baseline?

- **A.** It is the same symmetric hump, just shifted toward a slightly higher average in-degree.
- **B.** Many channels have low in-degree while a few have very high in-degree — a heavy tail.
- **C.** Every channel ends up with almost exactly the same in-degree as every other channel.
- **D.** Channels split into two equal groups, one of low in-degree and one of high in-degree.

**Answer: B**

**Reference:** Lecture 117 – Power Law emerges in WWW graphs (≈03:09–end)

**Solution:** A power-law distribution has a great many low-degree nodes and a small number of very-high-degree hubs, so its plot drops steeply and then trails off in a long tail — quite unlike the symmetric hump of the random model. It is not a shifted hump, not all-equal degrees, and not a two-group split.

**Q2.** [MCQ] In the baseline where channels recommend each other at random, why does the degree distribution form a bell-shaped hump?

**A.** Because a central planner assigns each channel a degree close to the chosen average.
**B.** Because the few hub channels force every other channel to copy their in-degree exactly.
**C.** Because random graphs are guaranteed to make every channel a hub before long.
**D.** Because each channel's degree is a sum of many independent random recommendations.

**Answer: D**

**Reference:** Lecture 116 – Why do Normal Distributions Appear? (≈06:25–end)

**Solution:** When an outcome is the sum of many independent random contributions, the result tends toward a normal (bell) distribution — the central-limit idea. With random recommendations, each channel's degree is just such a sum, so the distribution humps around the average. No planner, no hub-copying, and random graphs do not turn everyone into a hub.

---

**Q3.** [Match the following] Match each term in Column A with its description in Column B.

| Column A (term) | Column B (description) |
|---|---|
| 1. Normal distribution | a. a node whose degree is far larger than almost all others |
| 2. Power law | b. new nodes attach in proportion to existing degree |
| 3. Preferential attachment | c. a symmetric hump peaking near the average value |
| 4. Hub | d. many low-degree nodes and a few very-high-degree nodes |

**A.** 1-c, 2-d, 3-a, 4-b
**B.** 1-d, 2-c, 3-b, 4-a
**C.** 1-c, 2-d, 3-b, 4-a
**D.** 1-c, 2-a, 3-b, 4-d

**Answer: C (1-c, 2-d, 3-b, 4-a)**

**Reference:** Lectures 115–119 – Normal vs power law, preferential attachment

**Solution:** A normal distribution is the symmetric hump near the average (1-c); a power law has many small-degree nodes and a few huge ones (2-d); preferential attachment links new nodes in proportion to existing degree (3-b); and a hub is a node of exceptionally high degree (4-a).

---

**Q4.** [MCQ] What is the preferential-attachment ("rich-get-richer") rule that Anjali uses to explain the shape?

**A.** A new channel is recommended in proportion to how high a channel's existing degree already is.
**B.** A new channel is recommended only by the channels that currently have the lowest degree.
**C.** A new channel is recommended by every existing channel with exactly equal probability.
**D.** A new channel is recommended only after every existing channel has the same degree.

**Answer: A**

**Reference:** Lecture 119 – Rich Get Richer Phenomenon (≈00:12–end)

**Solution:** Under preferential attachment a newcomer attaches to existing nodes with probability proportional to their current degree, so well-connected channels keep gaining more — the rich get richer. It is not biased toward the lowest-degree nodes, not uniform across all nodes, and does not wait for degrees to equalise.

---

**Q5.** [MCQ] To confirm the pattern, Anjali makes the plot below. A straight line appears. What does that indicate?

```
import numpy as np, matplotlib.pyplot as plt
K, fK = degree_distribution(G)      # degree K and its fraction f(K)
plt.plot(np.log(K), np.log(fK))     # log-log plot
```

**A.** A bell-shaped normal distribution, since the log-log plot has curved away from a line.
**B.** Nothing at all; taking logarithms hides whatever structure the data may have had.
**C.** That every channel shares one common degree, shown by the single straight line.
**D.** A power law $f(K) \propto 1/K^{\alpha}$, whose exponent $\alpha$ is the slope's magnitude.

**Answer: D**

**Reference:** Lecture 118 – Detecting the Presence of Power Law (≈03:35–end)

**Solution:** Taking logs of $f(K) = 1/K^{\alpha}$ gives $\log f(K) = -\alpha \cdot \log K$, the equation of a straight line of slope $-\alpha$. So a straight line on the log-log plot is the litmus test for a power law, and the slope's magnitude is the exponent $\alpha$. A normal distribution would not straighten this way, the log does not hide structure, and it does not mean a single shared degree.

---

**Q6.** [Statement type] Consider the two statements about power-law degree distributions and decide their truth.

Statement I: In a power-law degree distribution, a small number of nodes have very high degree.
Statement II: A power-law degree distribution is a symmetric bell curve about its average.

**A.** Both statements are true.
**B.** Both statements are false.
**C.** Statement I is true, but Statement II is false.
**D.** Statement I is false, but Statement II is true.

**Answer: C**

**Reference:** Lecture 117 – Power Law emerges in WWW graphs

**Solution:** The first statement is correct: a power law has a few very-high-degree hubs in its long tail. The second is wrong: a power law is strongly skewed with a long tail, not a symmetric bell curve — that symmetric shape describes the normal distribution instead. So the first holds and the second fails.

---

**Q7.** [MCQ] Concretely, how is the litmus test for a power law carried out, and what does the slope mean?

**A.** Plot f(K) against K directly; a rising straight line confirms a power law of slope $+\alpha$.
**B.** Plot f(K) against log K; any curve at all confirms the network follows a power law.
**C.** Plot K against f(K); a flat horizontal line confirms a power law of exponent zero.

**D.** Plot log f(K) against log K; a straight line of slope −α signals f(K) ∝ 1/K^α.

**Answer: D**

**Reference:** Lecture 118 – Detecting the Presence of Power Law (≈06:02–end)

**Solution:** Because log f(K) = −α·log K, plotting log f(K) against log K yields a straight line whose slope is −α; that straightness is what confirms the power law. A plain (non-log) plot does not straighten, an arbitrary curve confirms nothing, and a horizontal line would mean no dependence on K at all.

---

**Q8.** [MCQ] In Anjali's random-recommendation baseline with 1000 channels and an edge probability of 0.1, around which in-degree does the bell-shaped distribution peak?

**A.** About 1000, which is the full number of channels in the network.
**B.** About 100, the channel count times the edge probability.
**C.** About 500, since the edge probability is exactly one in ten.
**D.** About 10, the edge probability rewritten as a percentage value.

**Answer: B**

**Reference:** Lecture 115 – Introduction to Power Law (≈08:56–end)

**Solution:** For a random graph G(n, p) the degree distribution is a bell curve peaking near the mean degree n × p; here 1000 × 0.1 = 100. The full channel count, a halfway value, and the bare probability read as a percentage all misapply the rule.

---

**Q9.** [MSQ] Which statements about power-law networks are correct? (Select all that apply.)

I. A power-law degree distribution has a long tail of a few very-high-degree nodes.
II. Preferential attachment grows degree in proportion to a node's existing degree.
III. A log-log plot of the distribution is roughly a straight line.
IV. Most nodes in a power-law network have a degree close to the network's average.

**A.** I, II, IV
**B.** II, III, IV
**C.** I, II, III
**D.** I, III

**Answer: C (statements I, II, III)**

**Reference:** Lectures 117–119 – Power law, detection, preferential attachment

**Solution:** The long tail of hubs (I), the proportional growth rule (II), and the straight log-log signature (III) are all correct. “Most nodes near the average” describes a normal distribution, not a power law, so IV is false.

---

**Q10.** [MCQ] Anjali reads that the power-law exponent α is not always exactly 2. What is typically observed across real networks?

**A.** α is always exactly 1, identical for every real-world network ever measured.
**B.** α is usually negative, so the distribution rises rather than falls with degree.
**C.** α is always greater than 10, which is why the hubs end up so dominant overall.
**D.** α typically lies between 2 and 3, differing from one network to the next.

**Answer:** **D**

**Reference:** Lecture 118 – Detecting the Presence of Power Law (≈00:06–02:28)

**Solution:** Measured power-law exponents commonly fall somewhere between 2 and 3, differing across networks rather than sitting at one fixed value. It is not fixed at 1, not negative (the distribution falls with degree), and not as large as 10.

## Comprehension 2 — Building and Stress-Testing the Network in NetworkX

### Case Study

A developer implements three things in NetworkX: a preferential-attachment generator (the Barabási–Albert model), a purely random generator (the Erdős–Rényi model), and a routine that removes nodes to test how the network holds up. For the Barabási–Albert model the code starts from a small connected network of m0 nodes (built so every node has at least one edge) and then adds nodes one at a time, up to a target total n. Each new node is given m edges to existing nodes, with m kept at m0 − 1 so that m never exceeds m0.

The targets for each new node are chosen by preferential attachment. The code keeps a dictionary of degrees and, from it, builds a selection probability for each existing node that rises with that node's degree. To pick a node by these probabilities it builds a list of running (cumulative) totals and draws a random r in [0,1]: the chosen node is the first whose cumulative total is at least r, so nodes with larger probabilities occupy wider intervals and are picked more often. The Erdős–Rényi generator instead walks over every pair of distinct nodes and decides each possible edge by a single random draw against the user-supplied probability p. Finally, the stress test reads a real email network and repeatedly removes nodes until the graph is no longer connected counting how many removals that takes either picking nodes at random or always picking the current highest-degree node. Key fragments:

```
import networkx as nx, random

# Barabasi-Albert: per-node probability and cumulative list
degrees = dict(nx.degree(G))
prob = {v: degrees[v] / float(sum(degrees.values())) for v in G.nodes}
r = random.uniform(0, 1)        # pick the node whose cumulative prob >= r

# Erdos-Renyi: an edge for each pair with probability p
for i in G.nodes:
    for j in G.nodes:
        if i != j and random.random() <= p:
            G.add_edge(i, j)

# Attack: always remove the current highest-degree node
nodes = sorted(G.degree, key=lambda x: x[1], reverse=True)
G.remove_node(nodes[0][0])
```

### Questions 1–10

**Q1.** [MCQ] In the Barabási–Albert code, what are m0 and m, and how is m fixed?

- **A.** m0 is the final node count and m the edge probability; m is set to m0 × n each run.
- **B.** m0 is the initial network's size and m the edges per new node, with m set to m0 − 1.
- **C.** m0 and m are both the total number of nodes, fixed equal so the graph stays a tree.
- **D.** m0 is the edges per new node and m the initial size, with m always larger than m0.

**Answer: B**

**Reference:** Lecture 121 – Implementing the BA Model-1 (≈00:40–04:38)

**Solution:** m0 is the number of nodes in the small starting network and m is how many edges each new node brings; the code sets m = m0 − 1, which keeps the required condition m ≤ m0. m0 is not the final count, m is not a probability, and the two are not set equal.

---

**Q2.** [MCQ] How does the code compute a node's preferential-attachment probability?

**A.** Its degree is divided by the sum of all nodes' degrees, so higher degree means higher probability.
**B.** One divided by the total number of nodes, so every node is equally likely to be chosen.
**C.** Its degree multiplied by the total number of nodes, normalised at the very end of the run.
**D.** The reciprocal of its degree, so that low-degree nodes are the most likely to be chosen.

**Answer: A**

**Reference:** Lecture 121 – Implementing the BA Model-1 (≈04:38–end)

**Solution:** Each node's probability is its degree divided by the sum of all degrees, so a higher-degree node is proportionally more likely to receive the new edge — exactly the rich-get-richer rule. Equal probabilities, a degree-times-n product, or a reciprocal-of-degree rule would all break preferential attachment.

---

**Q3.** [MCQ] The cumulative-probability list for three nodes is shown. With r = 0.4, which node is connected to the new node?

```
cum = [['A', 0.2], ['B', 0.5], ['C', 1.0]]   # running cumulative probabilities
r = 0.4                                       # random.uniform(0, 1)
# choose the first node whose cumulative probability >= r
```

**A.** Node A, whose cumulative value 0.2 is the closest one below r.
**B.** Node C, because its cumulative value 1.0 is the largest in the list.
**C.** No node is chosen, because r = 0.4 does not equal any listed value.
**D.** Node B, since 0.2 < 0.4 ≤ 0.5 places r inside node B's interval.

**Answer: D**

**Reference:** Lecture 121 – Implementing the BA Model-1 (≈05:40–end)

**Solution:** The chosen node is the first whose cumulative probability is at least r. Node A's 0.2 is below 0.4, but node B's 0.5 is the first value ≥ 0.4, so r falls in node B's interval (0.2, 0.5] and node B is selected. It is not the largest value that wins, and r need not match a listed value exactly.

---

**Q4.** [Arrange in order] Arrange the steps of the Barabási–Albert generator in the correct order.

*Steps (in scrambled order):*

(i) For the new node, build each existing node's probability from its degree.
(ii) Start from a small connected network of m0 nodes.
(iii) Draw r in [0,1] and pick m distinct targets via the cumulative list.
(iv) Add a new node and connect it to those m chosen targets.
(v) Form the running cumulative-probability list of the existing nodes.

**A.** ii → i → v → iii → iv
**B.** i → ii → v → iii → iv

**C.** ii → v → i → iii → iv

**D.** ii → i → iii → v → iv

**Answer: A (ii → i → v → iii → iv)**

**Reference:** Lectures 121–122 – Implementing the BA model

**Solution:** Begin with the small m0-node network (ii), compute each existing node's probability from its degree (i), turn those into a running cumulative list (v), draw r and select m distinct targets (iii), and finally add the new node joined to those targets (iv).

---

**Q5.** [MCQ] In the Erdős–Rényi generator, how is each possible edge decided, and what does p control?

**A.** For each pair, draw r and add the edge when r ≤ p; a larger p yields more edges.

**B.** Edges are added only between the two highest-degree nodes; p sets how many rounds run.

**C.** Every pair is always joined; p merely decides the colour used when the graph is drawn.

**D.** A fixed count of exactly p edges is added in total, regardless of the number of nodes.

**Answer: A**

**Reference:** Lectures 123–124 – Implementing the ER model

**Solution:** For each of the pairs the code draws r and adds the edge when r ≤ p, so a larger p yields more edges (p = 1 gives a complete graph, p = 0 an empty one). It is not restricted to high-degree nodes, not every pair is joined, and p is a probability, not a fixed edge count.

---

**Q6.** [Statement type] Consider the two statements about the degree distributions the two generators produce, and decide their truth.

Statement I: The Barabási–Albert (preferential-attachment) network has a power-law degree distribution.

Statement II: The Erdős–Rényi (uniform-random) network also has a power-law degree distribution.

**A.** Both statements are true.

**B.** Both statements are false.

**C.** Statement I is true, but Statement II is false.

**D.** Statement I is false, but Statement II is true.

**Answer: C**

**Reference:** Lectures 122, 124 – Degree distributions of BA and ER networks

**Solution:** The first statement is correct: preferential attachment yields a power-law degree distribution. The second is wrong: adding edges uniformly at random gives a normal (bell-shaped) distribution, not a power law. So the first holds and the second fails.

---

**Q7.** [MCQ] In the attack routine, how is the “highest-degree” node found before removal?

**A.** By picking one node uniformly at random with random.choice over the node list.

**B.** By sorting the nodes by degree in descending order and taking the first one.

**C.** By sorting the nodes by degree in ascending order and taking the first one.

**D.** By removing whichever node was added most recently to the network.

**Answer: B**

**Reference:** Lecture 125 – Forced Versus Random Removal of Nodes (≈10:30–end)

**Solution:** The code sorts the (node, degree) pairs by degree in descending order and removes the first entry — the current highest-degree node. Ascending order would remove the lowest-degree node, random.choice is the other (random) strategy, and recency is not used.

---

**Q8.** [MCQ] What does the stress test reveal about random versus highest-degree removal?

**A.** On the real email network, removing hubs disconnects it in far fewer steps than random removal.
**B.** On the real email network, random removal disconnects it far faster than removing the hubs.
**C.** On a random ER network, removing hubs disconnects it in dramatically fewer steps than random.
**D.** On every network, the two strategies always take an identical number of removals to disconnect.

**Answer: A**

**Reference:** Lecture 125 – Forced Versus Random Removal of Nodes (≈12:43–end)

**Solution:** The email network is hub-bearing, so deliberately removing its highest-degree nodes disconnects it in very few steps, whereas random removal takes many more. On an Erdős–Rényi network, which has no hubs, the two strategies are similar — so the strategies are not identical everywhere.

---

**Q9.** [MCQ] How does the routine know when to stop removing nodes?

**A.** It stops after removing exactly half of the nodes, regardless of connectivity.
**B.** It keeps removing while nx.is_connected(G) holds, stopping once the graph is disconnected.
**C.** It stops as soon as the highest-degree node's degree first drops below the average.
**D.** It never stops; the loop must be interrupted manually by the user each time.

**Answer: B**

**Reference:** Lecture 125 – Forced Versus Random Removal of Nodes (≈04:00–07:06)

**Solution:** The loop continues while nx.is_connected(G) is true, removing one node per pass and counting the removals, and it halts the moment the graph becomes disconnected. There is no fixed half-the-nodes rule, no average-degree test, and no need for manual interruption.

---

**Q10.** [MSQ] Which statements about the implementations are correct? (Select all that apply.)

I. The BA model picks targets with probability proportional to existing degree.
II. The ER model adds each possible edge independently with probability p.
III. Removing high-degree nodes disconnects a hub-bearing network quickly.
IV. The ER model produces a power-law degree distribution like the BA model.

**A.** I, II, IV
**B.** II, III, IV
**C.** I, II, III
**D.** I, III

**Answer: C (statements I, II, III)**

**Reference:** Lectures 121–125 – BA, ER, and attack survivability

**Solution:** Proportional target-selection (I), independent per-pair edges with probability p (II), and rapid fragmentation of a hub-bearing network under hub removal (III) are all correct. The ER model gives a normal distribution, not a power law, so IV is false.

---